Oracle Machine Learning

Manage your models with Oracle Machine Learning on Autonomous Database

May 16, 2023 | 14 minute read



Mark Hornick
Senior Director, AI and Machine Learning Product Management

We know about the importance of model management to machine learning projects and that there are multiple approaches to ensure effective model management. Our topic today is, in the context of Oracle Autonomous Database, how does Oracle Machine Learning support model management?

Model Management

When talking about model management, the object central to our discussion is the *model*. A machine learning model is the representation of the patterns found in the data by a particular algorithm along with various metadata, like expected predictors, outputs, algorithm hyperparameters, and even model quality metrics. But model management must also include how the model was produced and what was used to produce it. So, let's start off our discussion with building models, then move on to deployment, followed by monitoring, and retraining models. We'll then cover versioning and model governance.

Build and store models

In the context of Oracle Machine Learning (OML), we can separate the model building discussion into building in-database models versus model objects produced using R and Python packages. You can build in-database models using the OML4SQL PL/SQL interface, as well as OML4Py and OML4R native Python and R APIs.

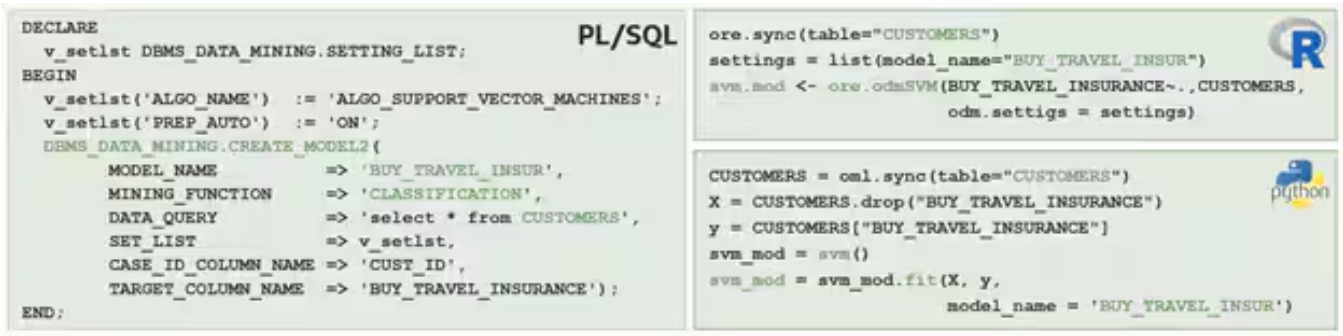


Figure 1: Build an in-database machine learning model to determine which customers are likely to buy travel insurance
using PL/SQL, R, or Python APIs.

Of course, you can build native Python and R models in local engines, or you can use OML Notebooks on ADB with custom-installed third-party packages.

In-database model storage

A key facet when building models involves where to store them. Your Autonomous Database (ADB) instance serves as a scalable centralized repository to ensure that everyone using that database can easily access models under database access controls. The in-database machine learning models exist as native first-class objects in the user schema, so there's no need to persist these separately.
Your models are also secure because you're leveraging the same security model that you trust your data to, and you can access your machine learning models just as easily as you access data from a table or view. Further, backup and recovery are also immediate as part of ADB.

R and Python model storage

In contrast, when using R and Python, models exist as objects in memory, and they normally need to be written out to files to save them. With OML, you store native R and Python models directly in your database instance using the datastore feature of OML4Py and OML4R. This eliminates the need to manage separate files and manually address backup, recovery, and security of these valuable objects. Furthermore, they can be quickly retrieved into R and Python engines for use with system-provided embedded execution or for local engine access. The following OML4Py code shows the code saving a model object in a datastore instance and loading it back into memory.

Copy code snippet

```
oml.ds.save(objs={'customer_segmentation_xgb':cs_xgb}, name='myDatastore')
oml.ds.load(name='myDatastore')
```

Deploy models

Model deployment is the bane of many machine-learning projects. For example, if you produce a model using R or Python, you need to make available an R or Python engine in which to run that model, and this often involves creating a container and provisioning a virtual machine in which to run it. Moreover, you need to ensure that the data can efficiently get to that engine and that results are delivered easily and securely. Given today's data volumes, these concerns can greatly complicate solution deployment if not render some solutions unable to meet SLAs.

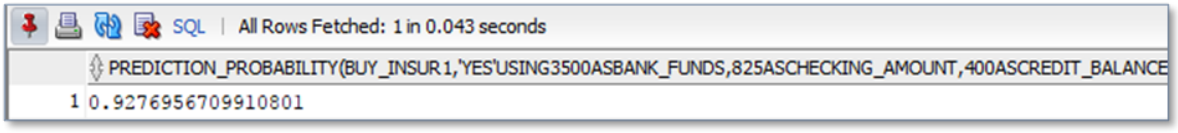
Deploy from the same database

With Oracle Machine Learning, your in-database models are immediately available after building, for example, by using a SQL query using the prediction operators built into the SQL language. Because model scoring – like model building – occurs directly in the database, there's no need for a separate engine or environment within which the model and corresponding algorithm code operate. Moreover, you can use models from a different schema (user account) if appropriate permissions are in place.

Here's an example using the model produced in Figure 1 of performing singleton and batch scoring. In the singleton case, we provide the data directly in the query.

Copy code snippet

```
SELECT prediction_probability(BUY_TRAVEL_INSUR, 'Yes' USING
   3500 as bank_funds, 1987 as cust_year_of_birth,
   'Married' as marital_status,
   2 as num_previous_cruises)
FROM dual;
```



In the batch scoring case, we can simply produce the result in a row set or choose to persist the result in a table using "create table as select" (CTAS). The following example also highlights getting prediction details to understand which predictor values most contributed to the individual prediction.

Copy code snippet

```
SELECT CUST_ID, round(BUY_INSURANCE_PRED,3) BUY_INSURANCE_PRED,
       RTRIM(TRIM(SUBSTR(OUTPRED."Attribute1",17,100)),'rank="1"/>') FIRST_A
       RTRIM(TRIM(SUBSTR(OUTPRED."Attribute2",17,100)),'rank="2"/>') SECOND_
FROM (SELECT CUST_ID, PREDICTION(BUY_TRAVEL_INSUR USING *) BUY_INSURANCE_PRE
             PREDICTION_DETAILS(BUY_TRAVEL_INSUR USING *) PD
      FROM TEST_DATA
      ORDER BY CUST_ID) OUT,
      XMLTABLE('/Details' PASSING OUT.PD COLUMNS
                "Attribute1" XMLType PATH 'Attribute[1]',
                "Attribute2" XMLType PATH 'Attribute[2]')  OUTPRED;
```

CUST_ID	BUY_INSURANCE_PRED	FIRST_ATTRIBUTE	SECOND_ATTRIBUTE
100005	1	"CUST_YEAR_OF_BIRTH" actualValue="1957" weight="2.496"	"YRS_RESIDENCE" actualValue="5" weight="1.696"
100009	0	"CUST_YEAR_OF_BIRTH" actualValue="1974" weight="3.291"	"Y_BOX_GAMES" actualValue="1" weight="2.209"
100010	0	"CUST_YEAR_OF_BIRTH" actualValue="1975" weight="2.357"	"YRS_RESIDENCE" actualValue="3" weight="1.338"
100012	1	"CUST_YEAR_OF_BIRTH" actualValue="1968" weight="1.144"	"YRS_RESIDENCE" actualValue="4" weight=".355"

Deploy to a different database

In-database models are portable across database instances. For example, users can build models in one database and deploy them in another, including on premises database, Database Cloud Service, and Oracle Autonomous Database. This enables having separate development, test, and production databases.

Alternatively, the code used to produce the model can simply be run in the target database to rebuild the model, e.g., in the production environment using the latest data, and then immediately used. See this blog post on deployment architecture alternatives involving OML.

Deploying R and Python models

OML also supports deploying native R and Python models. These can be stored in the database datastore as noted above and then used through embedded execution with OML4R and OML4Py. This embedded execution automatically spawns and manages R and Python engines. Embedded execution enables invoking R and Python user-defined functions from SQL and REST interfaces, which makes integration with most applications and dashboards quite convenient. The use of third-party packages is supported through conda environments.

Scalability

Of course, part of deploying any solution involves ensuring that it can scale to meet the demands of the enterprise. In-database models have proven to scale very well. For example, see this blog post on scoring performance factors and sample results.

For native R and Python models, the embedded execution feature of OML4Py and OML4R has system-supported data-parallel and task-parallel invocation. For example, a native Python scikit-learn SVM model can be invoked using multiple Python engines, where the database environment spawns and manages these engines, loads the user-defined Python function that performs the scoring and a "chunk" of data to each of these engines, while returning the result to the database – all automatically. If you have 100 million rows of data to score, you might ask for 10 Python engines, each to process 1 million rows of data at a time, in parallel. This achieves scalability, since all the data does not need to be loaded into memory at once, and performance, by being able to leverage multiple Python engines in parallel, while using third-party software.

Security

Since models exist in the database and are authenticated using the same database credentials that you trust the rest of your data to, your model scoring is just as secure. There's no need to provide other authentication and security measures to ensure that security is maintained.

This is the same for in-database and, when used with embedded execution, native R and Python models.

Deploy as lightweight REST endpoints

In support of real-time scoring use cases, OML Services supports deploying models as lightweight REST endpoints using a REST API. You can also deploy models to OML Services using the no-code Models UI, as shown in Figure 2, or OML AutoML UI. OML Services also supports asynchronous batch scoring using REST endpoints. See this blog post for more details on batch scoring.

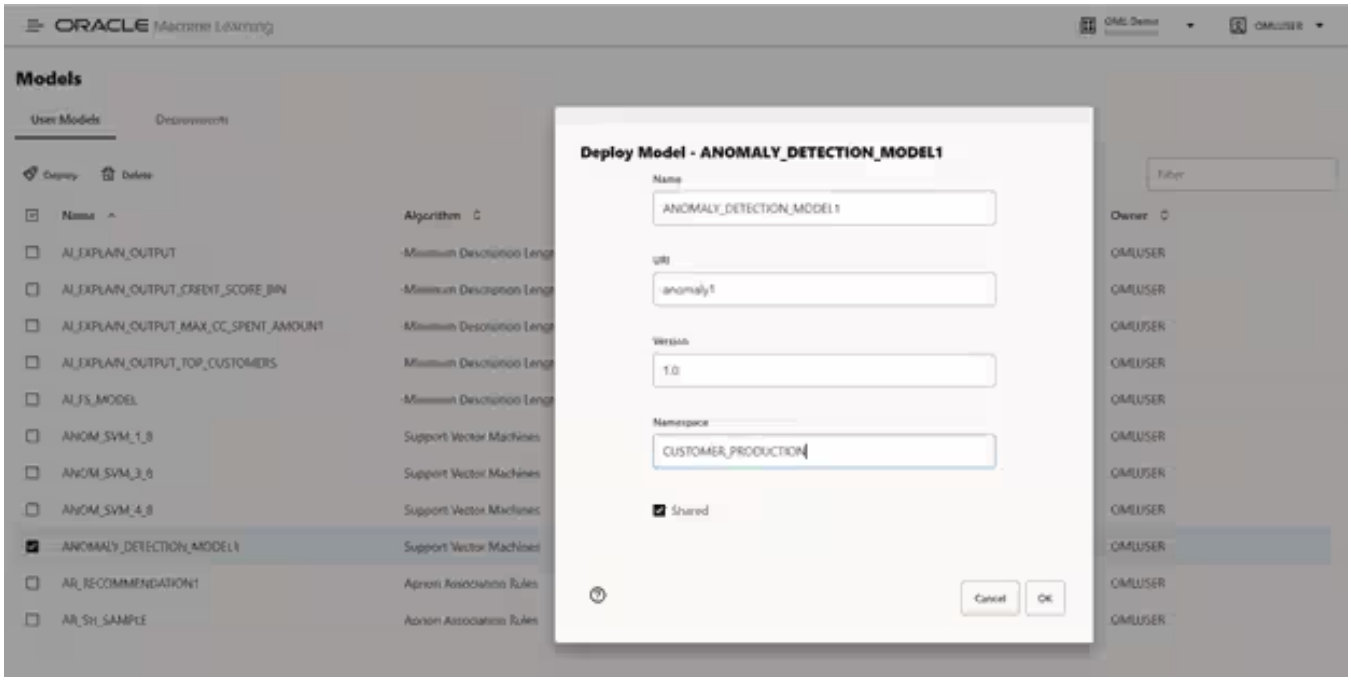


Figure 2: Version and deploy an in-database model to OML Services for scoring via REST endpoints.

Monitor models

While you may think that creating and deploying a model is the final objective, that's often just the beginning. A model that's placed in production needs to be monitored to ensure that it continues to meet the project success criteria. This involves tracking performance metrics or looking for changes in the distribution of the predicted classes and the data that's being presented to the model for scoring. When any of these quality metrics pass a predetermined threshold, users need to be notified to take corrective action. This may include rebuilding the model on newer data, reassessing the data that's being used including individual predictors and associated transformations, or choosing a different algorithm or combination of algorithms to meet project objectives.

As part of OML Services, both data monitoring and model monitoring are supported through a REST interface, with no-code user interfaces on the way. You specify a baseline data set and, for model monitoring, a model. Then you periodically provide additional data against which to assess the model, which can be configured to occur automatically.

Data monitoring provides insight into how data evolves over time – looking for potential data drift, which can occur abruptly or gradually. Data monitoring can discover data quality issues as well as shifts in feature interactions. Data monitoring can also help explain model quality degradation over time.

Model monitoring includes identifying when a given model metric, like balanced accuracy or mean squared error, significantly changes – or the distribution of predicted values deviates too much from initial values. Such deviations can signify the need to rebuild models. Another model monitoring technique involves concept drift, that is, when the statistical properties of the target variable change over time, again potentially suggesting the need to rebuild the corresponding model.

For additional details, see this blog post on OML Services Monitoring, and specifically data monitoring, and model monitoring.

Rebuild models

We can explore rebuilding models in the context of scheduling, data access and filtering, and redeploying models.

Scheduling jobs

As we noted above, our model monitoring and data monitoring may indicate the need to retrain models. Some projects may choose to retrain models on a set schedule. You can use notebooks to contain the complete code required to prepare data and build and evaluate resulting models. OML Notebooks allows you to schedule these notebooks to run once or on a recurring schedule—using a simple no-code user interface. Your notebooks may contain SQL, R, and Python code to prepare/explore data, build/evaluate models, and score data in batch. This approach supports both in-database and native R and Python models.

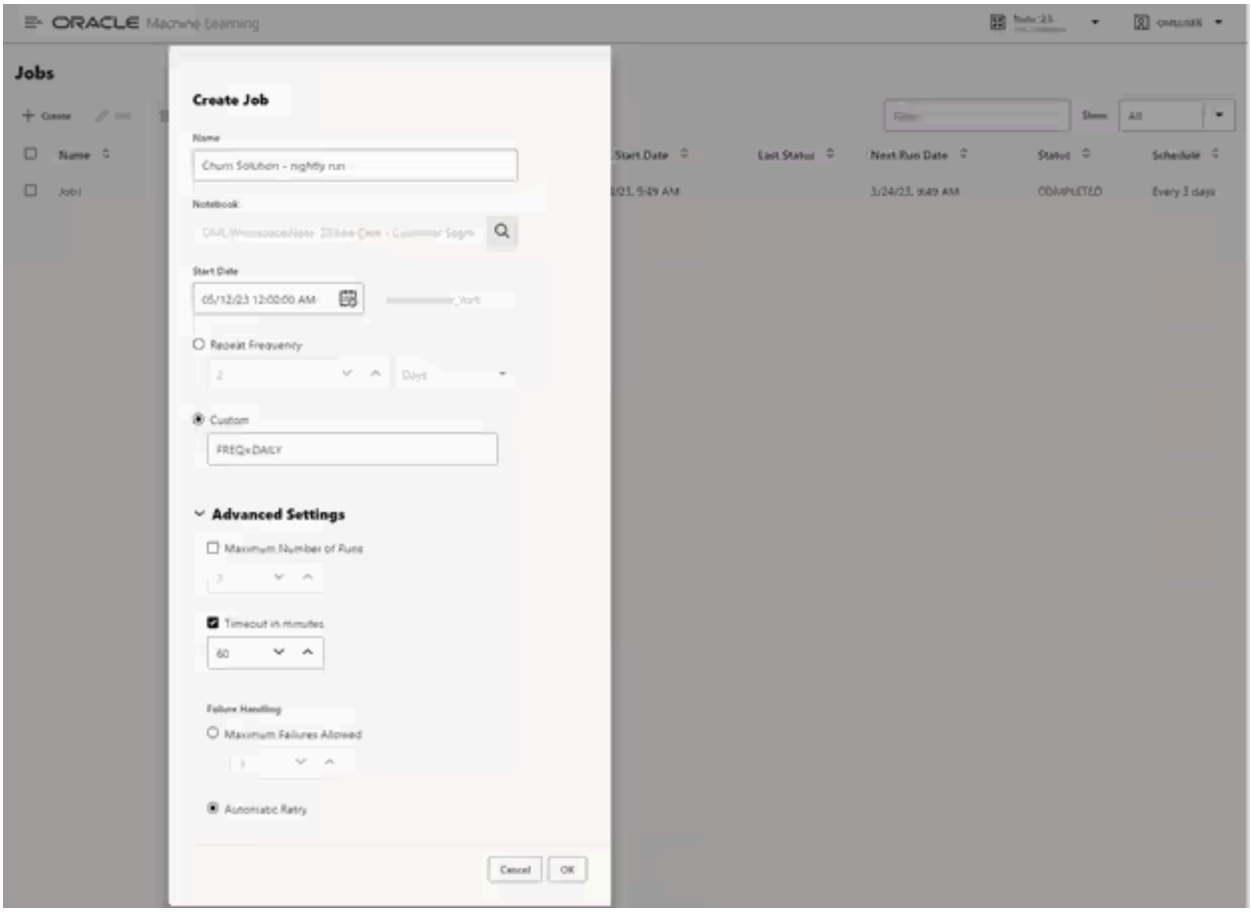


Figure 3: Using OML Jobs interface to schedule a Customer Segmentation notebook to run daily at midnight.

ADB also provides the DBMS_SCHEDULER package, which provides fine-grained and sophisticated control for scheduling jobs using PL/SQL, working seamlessly with OML4SQL. It can also be used with the SQL API when invoking Python or R user-defined functions via embedded execution.

Accessing and filtering data

When using third-party tools, the issue of collecting the next set of data for model retraining can be a challenge. However, if your model is built from one or more database tables, there's no need to collect data from various sources to rebuild that model. It's immediately available.

Further, because data can be specified as a SQL query to be used for retraining, that query can be specified to use the latest window of data available in a table. For example, we may want to use the last 90 days of customer transactions when building an association rules model. For example, the following

data query could be used when building a model, whether directly from SQL using CREATE_MODEL2, or via an R or Python proxy object and the specific in-database algorithm functions.

Copy code snippet

```
SELECT * FROM CUSTOMER_TRANSACTIONS where transaction_date > sysdate-30;

Python> TRAINING_DATA = oml.sync(query = "SELECT * FROM CUSTOMER_TRANSACTION

R> ore.sync(query = "SELECT * FROM CUSTOMER_TRANSACTIONS where transaction_d
```

This works for in-database models as well as third-party models.

Redeploy models

Rebuilding a replacement production model implies that this model will be redeployed once approved (either automatically or manually). Since model names within a schema must be unique, the rebuilt model will be given a new name, and after evaluation, the original model can be dropped (or exported and archived) and the new model renamed accordingly.

Copy code snippet

```
BEGIN
  DBMS_DATA_MINING.RENAME_MODEL(
    model_name      => 'census_model_v7',
    new_model_name  => 'census_model');
END;
```

To retain previous models—the preferred approach for model management—each in-database model take on a unique name. Applications using a given model may use a configuration parameter to specify the currently approved model. By retaining previous models, you can quickly switch back to a previous model, or use multiple models side-by-side a trial period, such as A/B testing.
When rebuilding native R and Python models, the datastore serves as the repository. A datastore naming approach—like the in-database models—can be used to rebuild previous R and Python models, where the application also uses a configuration parameter based on the datastore name, or object name within a given datastore.

Version models

When we talk about model versioning, there are a few facets to consider: the model objects themselves, whether they are in-database or R/Python models, and the code that produces those models. Let's start with in-database model objects.

In-database model objects

To version a model itself, OML supports versioning on ADB through OML Services, which allows you to track each model deployment. The OML Models user interface facilitates deploying in-database models and viewing their metadata in a browser window, but the OML Services REST interface also enables programmatic interaction. OML Services also supports deploying third-party ONNX-format models via the REST API.

Alternatively, since in-database models exist in your database schema, they can be backed up or archived along with other schema objects—like database tables or views—in your file system and versioned using your enterprise-sanctioned versioning system.

R and Python model objects

In the case of native R and Python models, these can be stored in the database datastore using a suitable naming convention. For example, if we have a native XGBoost model in a variable cs_xgb in Python, we can store it in the datastore using the following convention:

Copy code snippet

```
oml.ds.save(objs={'customer_segmentation_xgb':cs_xgb},
            name='/team1/project/customer_segmentation/xgboost_model/v1',
            description = "XGBoost model supporting customer segmentation pr
```

Subsequent versions of the same model can be included in datastores named “vN”, where N is the version number. Then, loading objects back into memory would use the following:

Copy code snippet

```
oml.ds.load(name='/team1/project/customer_segmentation/xgboost_model/v1',
            objs=['customer_segmentation_xgb'])
```

The resulting object is then available for use in memory.

Notebook versioning

The ability to version the code and process used to derive a model can be just as important, if not more important, as the model itself.
Using OML Notebooks on ADB, users can version notebooks that include data preparation, model building and evaluation, and depending on the use case, even deployment. Since the results of running notebook paragraphs remain in the notebook, users have a record of a given run, which is also versioned with the notebook. Figure 4 shows the OML Notebooks EA edition that allows versioning notebooks while editing and comparing notebook versions.

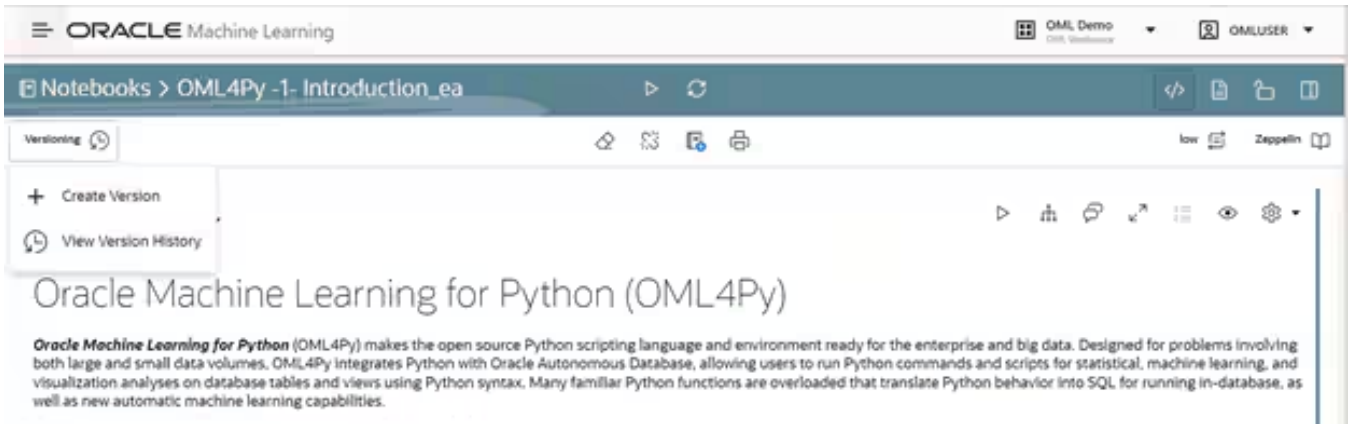


Figure 4: Use OML Notebooks EA to create a new version of a notebook.

Documentation

As with any software related project, it's essential to document code, not only to help teammates understand what you've done, but to help you remember what you did. In the case of machine learning projects, we need to document all phases of the machine learning process – from project definition and success criteria through data preparation, modeling, evaluation, and deployment. For example, what's the project objective, what's the purpose of the model, how do we interpret model inputs and outputs, what is the required data preparation, how do we rebuild the model, and any other contextual information that helps the organization use or maintain models and ML-based solutions.

OML Notebooks

By leveraging OML Notebooks, users have access to markdown to supplement code paragraphs with documentation on all aspects of the ML project, including data preparation and modeling. Notebooks also have associated comments that are visible in the notebook listing for ease of locating relevant content.
Notebooks can also be shared as read-only templates for use by the data science team or broader organization with associated comments and tags to facilitate understanding and finding relevant content.

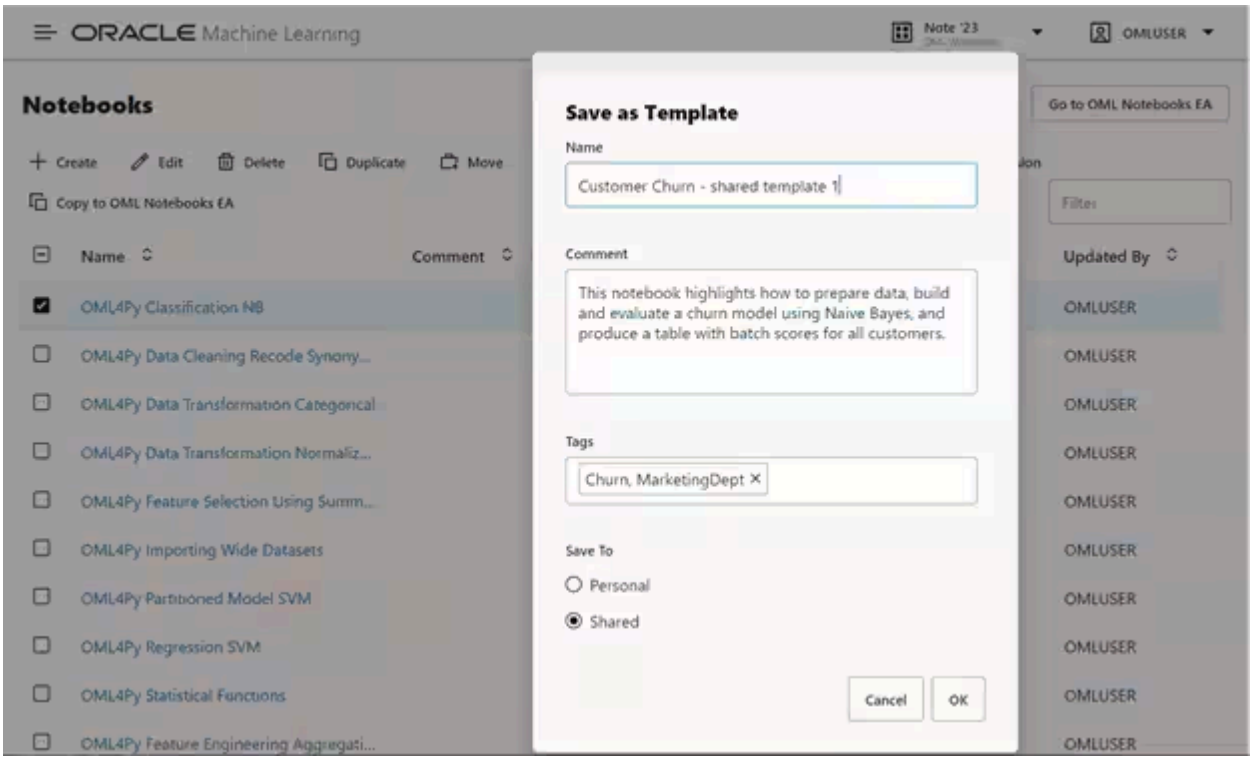


Figure 5: Save a notebook as a template for sharing with other users. You can instantiate a copy of this template notebooks to run or edit it.

Model objects, tables, views, and columns

On the individual model front, OML in-database models allow you to associate comments with individual model objects as shown here using the SQL interface:

Copy code snippet

```
COMMENT ON MINING MODEL schema_name.model_name IS string;
COMMENT ON MINING MODEL dt_sh_clas_sample IS
```

```
            'Decision Tree model predicts promotion response';
SELECT model_name, mining_function, algorithm, comments FROM user_mining_moc

MODEL_NAME         MINING_FUNCTION  ALGORITHM      COMMENTS
------------------ ---------------- -------------- -------------------------
DT_SH_CLAS_SAMPLE CLASSIFICATION   DECISION_TREE  Decision Tree model predic
```

You can also document tables and their columns:

Copy code snippet

```
COMMENT ON TABLE employees IS 'Organization A employees';
COMMENT ON COLUMN employees.job_id IS 'abbreviated job title';
```

Model metadata

Implicitly available with each model are associated model detail views, which serve as system-provided documentation. These include, for example the model hyperparameters (settings) and other content that can be algorithm specific. For example, for a neural network model named "fleet_predictive_maintenance", we can obtain the set of model views using the following query:

Copy code snippet

```
SELECT VIEW_NAME, VIEW_TYPE
FROM USER_MINING_MODEL_VIEWS
WHERE MODEL_NAME=' fleet_predictive_maintenance'
ORDER BY VIEW_NAME;
```

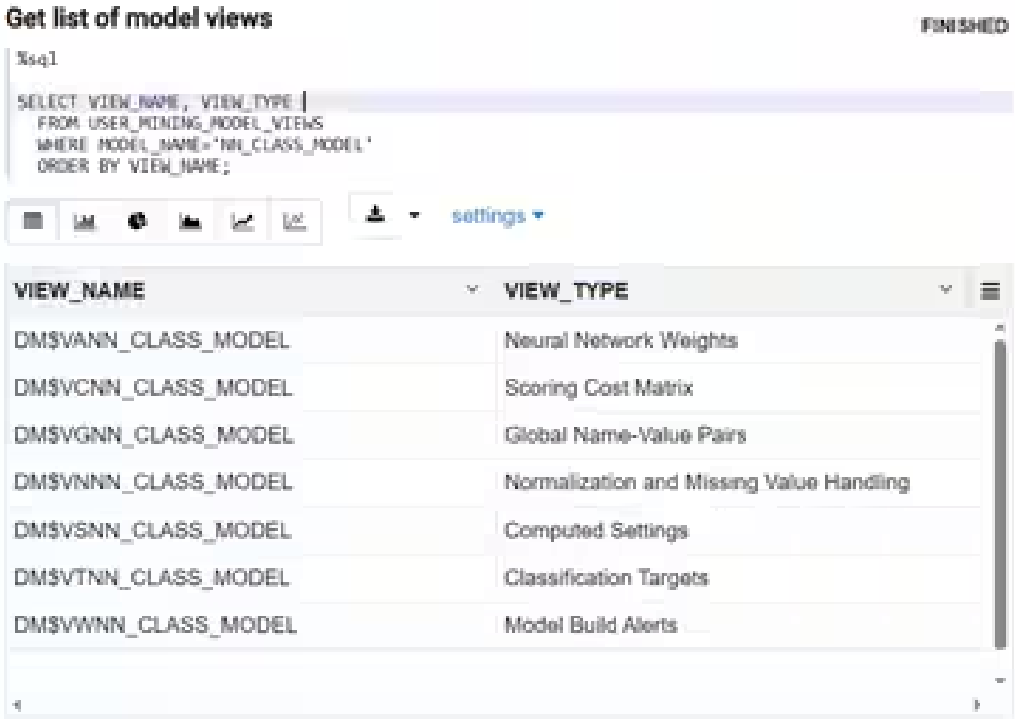


Figure 6: Model metadata available through model detail views.

In Figure 6, we see that model views include neural network weights, the scoring cost matrix, data preparation actions, and computed settings, among others.

Model governance

One aspect of model governance involves clear ownership of models and data. Using database authentication, each user may have their own schema (user account) and be granted privileges to access data as well as build, modify, and use/deploy models. User actions can also be tracked by the database for auditing purposes. Model creation and access can be audited, where access involves scoring data or exploring model detail views.

Regarding data and model security, ADB provides access controls to allow administrators to ensure data privacy and security, as well as model security. As all these objects exist in the database environment, the same safeguards afforded your data are also afforded your models.

Models also need to offer transparency or explainability so that the predictions made can be properly audited or evaluated to ensure fairness and safeguard against bias. To this end, OML supports "prediction details" such that you can understand which predictors most contribute to an individual prediction, along with a corresponding weight for each of those predictors. At the model level, users can also assess the relative importance of all the predictors for a given model. This helps you to understand which factors are most likely to influence the overall model.

Summary

Model management is essential to successful machine learning projects. Oracle Machine Learning provides support for model management across multiple dimensions as described above. We've explored this in the context of Oracle Autonomous Database, but many of these capabilities also apply to Oracle Database.

For more information, see the OML product documentation.



Mark Hornick
Senior Director, AI and Machine Learning Product Management

Mark Hornick is Senior Director, AI and Machine Learning Product Management. Mark has more than 20 years of experience integrating and leveraging AI/machine learning with Oracle software as well as working with internal and external customers to apply Oracle's machine learning
Show more

Previous post
ICYMI: Embedding AI/ML in your application using Oracle Machine Learning

Next post
Announcing the release of Oracle Machine Learning for Python 2.0

Mark Hornick | 2 minute read

on Autonomous Database

Mark Hornick | 2 minute read

Resources for

About
Careers
Developers
Investors
Partners
Startups

Why Oracle

Analyst Reports
Best CRM
Cloud Economics
Corporate Responsibility
Security Practices

Learn

What is Customer Service?
What is ERP?
What is Marketing Automation?
What is Procurement?
What is Talent Management?
What is VM?

What's New

Try Oracle Cloud Free Tier
Oracle Sustainability
Oracle COVID-19 Response
Oracle and SailGP
Oracle and Premier League
Oracle and Red Bull Racing Honda

Contact Us

US Sales 1.800.633.0738
How can we help?
Subscribe to Oracle Content
Try Oracle Cloud Free Tier
Events
News

© 2026 Oracle Privacy / Do Not Sell My Info Cookie Preferences Ad Choices Careers